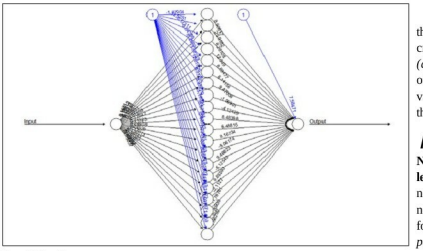

Figure 3: Neural net

Rendering R as XML (portable data format): With the increasing prevalence of Service Orientation Architecture (SOA) and cloud services like SaaS and Paas, we are very often interested in exposing our results via common information sharing platforms like XML or Jason. R provides an XML package to do all the tricks and tradeoffs with XML. The following snippet describes XML loading and summarisation:

```
>install.packages("XML");
>library(XML);
>dat <- xmlParse("URL")
> xmlToDataFrame(getNodeSet(dat, "//value"));
```

Use the actual URL of the XML file in place of *'URL'* in the third statement. The next statement lists compatible data in row/column order. The node name is listed as the header of the respective column while its values are listed in rows.

A more reasonable approach in R is exposing results as XML. The following scenario does exactly this. Let there be two instances of probability distribution, namely, the current and the previous one for two probability distribution functions —Poisson and Exponential. The code follows below:

```
>x1<-ppois(16, lambda=12);
>x2<- pexp(2, rate=1/3);
>y1<-ppois(16, lambda=10);
>y2<- pexp(2, rate=1/2);
>xn <- xmlTree("probability_distribution");
>xn$addTag("cur_val",close="F");
>xn$addTag("poisson",x1);
>xn$addTag("exponential",x2);
>xn$closeTag();
>xn$addTag("prev_val",close="F");
>xn$addTag("poisson",y1);
>xn$addTag("exponential",y2);
>xn$closeTag();
>xn$value();
```

Let's begin by creating a root node for the XML document. Then we go forward by creating child nodes using the method *addtag (addtag* maintains parent child tree structure of XML). Once finished with adding tags, the value preceded by the root node name displays the XML document in a tree structure.

Advance data analytics

Neural networks (facilitating machine learning): Before proceeding further, we need to power up our R environment for neural network support by executing the following command at the R prompt *install. packages("neuralnet")*; and load using command *library (neuralnet);* into the R environment. Neural networks are complex mathematical frameworks that often contain mapping of sigmoid with the inverse sigmoid function, and have sine and cosine in an overlapped state to provide timeliness in computation. To provide learning capabilities, the resultant equation is differentiated and integrated several times over different parameters depending on the scenario. (A differentiating parameter in the case of banking could be the maximum purchases made by an individual over a certain period—for example, more electronic goods purchased during Diwali than at any other time in the year.) In short, it may take a while to get the result after running a neural net. Later, we will discuss combinatorial explosion, a common phenomenon related to neurons.

For a simple use case scenario, let's make a neural network to learn about a company's profit data at 60 instances; then we direct it to predict the next instance, i.e., the 61st.

```
>t1 <-  as.data.frame(1:60);
>t2 <-  2*t1 - 1;
>trainingdata <- cbind(t1,t2);
> colnames(trainingdata) <- c("Input","Output");
> net.series <- neuralnet(Output~Input,trainingdata,
hidden=20, threshold=0.01);
> plot(net.series);
> net.results <- compute(net.series, 61);
```

In the above code, Statement 1 loads numeral 1 to 60 into t1, followed by loading of data into t2 where each instance = 2*t1-1. The third statement creates a training data set for the neural network, binding together the numerals with its instance, i.e., t2 with t1. Next, let's term t1 as input and t2 as output to make the neural network understand what is input into it and what it should learn. In Statement 5, actual learning takes place with 20 hidden neurons (the greater the number, the greater is the efficiency or lesser the approximation); .01 is the threshold value where the neuron activates itself for input. In Statement 6, we see a graphical plot of this learning, labelled by weights (+ and -). Refer to Figure 3. Now the